Ejercicio 2 (1,5 puntos). Sin definir otras funciones adicionales, implementar una función en lisp llamada **contarCondiciones** que, dada una **lista sin anidamientos de valores** y un **número arbitrario de funciones que representen condiciones**, devuelva una lista con el número de elementos de la lista de valores que satisfacen cada una de las condiciones.

(contarCondiciones '(1 2 3) (lambda(x)(> x 1))) → (2)

(contarCondiciones '(1 2 3 4 5) (lambda(x)(> x 1)) (lambda(x)(< x 1)) (lambda(x)(= x 1))) → (4 0 1)

Ejercicio 2 (1,5 puntos). Sin definir otras funciones adicionales, implementar una función en lisp llamada **primeroCentralyUltimo** que, dada **una lista sin anidamientos**, devuelva una lista de tres elementos compuesta por el primer elemento, el elemento central y el último elemento. Si el número de elementos es par, el elemento central será NIL.

(primeroCentralyUltimo '()) → (NIL NIL NIL)

(primeroCentralyUltimo '(1 3 4 5)) → (1 NIL 5)

(primeroCentralyUltimo '(1 3 8 4 5)) → (1 8 5)

Ejercicio 3 (1,5 puntos). Sin definir otras funciones adicionales **ni utilizar las funciones max y min**, implementar una función en lisp llamada **SumaMaxMin** que, dado **un número arbitrario de argumentos numéricos**, devuelva una lista de tres elementos con la suma, el máximo y el mínimo, en este orden.

(SumaMaxMin 1 2 3 3 4 5 8 6 0 4) → (36 8 0)

(SumaMaxMin 5) → (5 5 5)

Aritméticas

(+ s_1 s_2 s_3 ... s_n) (- s_1 s_2 s_3 ... s_n)
(* s_1 s_2 s_3 ... s_n) (/ s_1 s_2 s_3 ... s_n)
(1+ s) (1- s)
(MAX s_1 s_2 s_3 ... s_n) (MIN s_1 s_2 s_3 ... s_n)
(FLOOR s_1) (CEIL s_1)
(MOD s_1 s_2)

Listas

(LIST e_1 e_2 … e_n) Devuelve una lista formada por los elementos especificados
(CAR lista) Retorna el primer elemento
(CDR lista) Retorna la cola de la lista
(CONS elemento lista) Devuelve nueva lista cuyo primer elemento es elemento y cuya cola es lista
(APPEND $lista_1$ $lista_2$ … $lista_n$) Concatenación de listas
(REVERSE lista) Invertir los elementos
(LENGTH lista) Devuelve el número de elementos
(REMOVE elemento lista) Borra el elemento dado de la lista

Comprobación de tipos

(ATOM arg) – ¿Es un átomo? T y NIL lo son
(LISTP arg) – ¿Es una lista?
(NUMBERP arg) – ¿Es un número?

Comprobación de valor

(ZEROP arg) – Devuelve T si arg es 0
(MINUSP arg) – Devuelve T si arg es negativo
(NULL arg) – Devuelve T si arg es NIL

Comparación de valores

(EQUAL arg_1 arg_2)
(MEMBER Elemento Lista)
(< arg_1 arg_2 arg_3 … arg_n)
(>arg_1 arg_2 arg_3 … arg_n)
(=arg_1 arg_2 arg_3 … arg_n)

Predicados lógicos

(NOT s)
(AND s_1 s_2 s_3 ... s_n)
En el caso de evaluar todos a verdadero, devuelve el último valor obtenido (el de más a la derecha)
(OR s_1 s_2 s_3 ... s_n)
En el caso de evaluar a verdadero, devuelve el primer valor distinto de NIL obtenido (el de más a la izquierda)

Estructuras condicionales

(IF Condicion ResultadoSiT ResultadoSiNoT)

```
(COND   (Condicion1 Resultado1)
        (Condicion2 Resultado2)
        …
        (Condicionn Resultadon))
```

Si ninguna condición evalúa a verdadero devuelve NIL
Para hacer condiciones exhaustivas, la $Condición_n$ puede ser T

Definición de funciones

(DEFUN NombreFuncion(s_1 s_2 s_3 … s_n) Descripción)

(lambda (x) (* 2 x))

Llamadas a funciones

(funcall *función* s_1 s_2 s_3 … s_n)
(apply *función* s_1 s_2 s_3 … s_n)
(*NombreFuncion* s_1 s_2 s_3 … s_n)

Obtención de función a partir del nombre:

#'nombre o FUNCTION nombre